The Journal of Supercomputing (2023) 79:2082–2107
https://doi.org/10.1007/s11227-022-04717-8

Traffic-aware dynamic controller placement in SDN using NFV

G. Ramya[1] · **R. Manoharan**[1]

Accepted: 11 July 2022 / Published online: 6 August 2022
© The Author(s), under exclusive licence to Springer Science+Business Media, LLC, part of Springer Nature 2022

Abstract

Software-Defined Networking (SDN) and Network Function Virtualization (NFV) are promising technologies for delivering software-based networks to the user community. The application of Machine Learning (ML) in SDN and NFV enables innovation and easiness towards network management. The shift towards the softwarization of networks broadens the many doors of innovation and challenges. As the number of devices connected to the Internet is increasing swiftly, the SDNFV traffic management mechanism will provide a better solution. Many ML techniques applied to SDN focus more on the classification problems like network attack patterns, routing techniques, QoE/QoS provisioning. The approach of the application of ML to SDNFV and SDN controller placement has a lot of scope to explore. This work aims to develop an ML approach for network traffic management by predicting the number of controllers likely to be placed in the network. The proposed prediction mechanism is a centralized one and deployed as Virtual Network Function (VNF) in the NFV environment. The number of controllers is estimated using the predicted traffic and placed in the optimal location using the K-Medoid algorithm. The proposed method is suitably analysed for performances metrics. The proposed approach effectively combines SDN, NFV and ML for the better achievement of network automation.

Keywords SDN · NFV · Network management · SDN controller · Number of controllers · Traffic prediction

✉ G. Ramya
ramya028@pec.edu

[1] Department of Computer Science and Engineering, Pondicherry Enginnering College, Puducherry 605014, India

1 Introduction

The proliferation of network devices and network applications being deployed demands fast and reliable connectivity. As the number of IoT applications is increasing rapidly, the number of devices connected to the Internet is predicted to be 29.3 billion by the end of 2023 [1]. Therefore, more flexible, adaptable and automated network environment to deploy new functionalities becomes essential.

The major issues with the traditional network are (1) tightly coupled network architecture, (2) proprietary-dependent devices, (3) time-consuming process to add/remove network devices and (4) increase CAPEX and OPEX with respect to network size. The current network trend demands flexibility, agility and multi-tenancy, whereas the new technologies offer various applications such as automated driving and many other applications that needs an effective architecture that is highly scalable, supports automation and high-end virtualization. Recently, the SDN and the NFV are two promising technologies that may provide a solution to the current challenges in the network. Both the technologies support network automation to a greater extent.

The SDN enables the programmability of the network devices by decoupling the control plane and data plane [2, 3]. The SDN architecture consists of three planes, viz., data, control and application planes. The data plane consists of forwarding devices, control plane consists of SDN controller, and application plane consists of SDN applications. The SDN planes communicate via programming interfaces. OpenFlow [2] is the widely used protocol to establish communication between the data and the control plane.

The NFV is a network paradigm which aims to reduce the number of middle boxes through softwarization of network functionalities [5]. Both SDN and NFV rely on the concept of abstraction of network resources, functionalities and moving towards programmable networks. Integrated SDNFV would create further opportunities for adaptive design and highly automated networking environments [5].

Recently, ML algorithms have attracted both the academics and industries with the variety of algorithms it provides [6]. In networking, many ML algorithms are applied to carry out traffic classification, to predict network patterns, to identify threats and many more [7]. The Venn diagram Fig. 1 shows the advantages of applying ML into SDNFV.

In SDN-based networks, the SDN controllers have a crucial role to play and act as a Network Operating System (NOS). The SDN controller is responsible for performing all control activities in the network. The SDN controller monitors, manages and maintains the global view of the network. The network performance is truly based on the controller's performance.

Whenever an incoming packet arrives at the switch end, the switch looks for its matching flow pattern in its flow table. If the matching flow value is present in the table, the switch performs forward/reject on it. In the absence of matching flow pattern, the switch sends the incoming packet as PACKET_IN message to the SDN controller to populate the flow table entry for the incoming packet. The

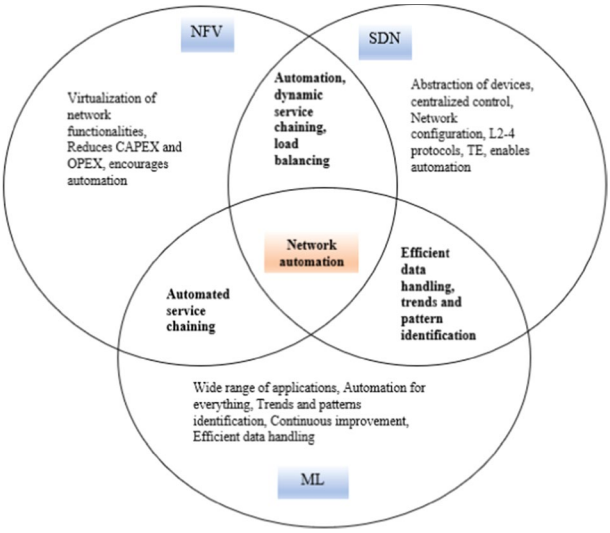


Fig. 1 Advantages of applying ML in SDNFV environment

SDN controller populates and sends the flow instructions as PACKET_OUT message to the switch to perform the specified action for the incoming packet [2, 3]. Therefore, the controller and the switches must be placed in a minimum distance to avoid packet processing latency.

In SDN-based networks, the SDN controllers have a crucial role to play and acts as a Network Operating System (NOS). The SDN controller is responsible for performing all control activities in the network. The SDN controller monitors, manages and maintains the global view of the network. The network performance is truly based on the controller's performance.

Whenever an incoming packet arrives at the switch end, the switch looks for its matching flow pattern in its flow table. If the matching flow value is present in the table, the switch performs forward/reject on it. In the absence of matching flow pattern, the switch sends the incoming packet as PACKET_IN message to the SDN controller to populate the flow table entry for the incoming packet. The SDN controller populates and sends the flow instructions as PACKET_OUT message to the switch to perform the specified action for the incoming packet [2, 3]. Therefore, the controller and the switches must be placed in a minimum distance to avoid packet processing latency.

According to Hellar [4], a single controller placement is sufficient to handle the entire network. But the growth of new technologies and the number of devices connected to the Internet limit the performance of single controller. Therefore, the idea of placing multiple controllers is introduced. The multiple controller placement opens up the issues like identification of number of controllers required and their optimal locations in the network.

Though the controller placements are based on the network topology, the placement becomes ineffective if that controller zone is not utilized properly or there is

no network flow. Therefore, a traffic-aware controller placement is needed to ensure proper utilization of controllers and to avoid unnecessary controller placements which helps in proper resource planning and utilization.

Considering the significance of traffic-aware optimal controller placements, this work aims to develop a ML-based traffic classification mechanism for SDNFV architecture. The proposed methodology identifies the number of controllers dynamically by analysing the incoming traffic and places them in the optimal locations in the network. The major contributions in this work are as follows:

1. Proposed a SDNFV architecture that addresses the Controller Placement Problem (CPP) in SDN.
2. Proposed a Long short-term memory (LSTM)-based network traffic classification and prediction algorithm to perform vTCP SDN functionalities.
3. Further, it deploys the vTCP SDN functionality as VNF and made it available to all the controllers placed in the network.
4. Finally, a greedy approach is proposed to identify the number of controllers to be placed in the network by using predicted traffic.

The rest of the paper is organized as follows. Related work is presented in Sect. 2. The problem statement is defined in Sect. 3, and the proposed mechanism is discussed in Sect. 4. The experimental setup and the obtained results are discussed in Sect. 5. This paper finally ends with the conclusions and future enhancements.

2 Related work

This section briefly explains the works done with respect to SDN controller placement, SDN traffic classification and works related to integrated SDNFV architecture.

2.1 SDN controller placement

The CPP has attracted many researchers to put forth their ideas to place controller(s) in the optimal locations in the network. Ref. [4] addressed the CPP for the first time. The authors studied the impact of number of controllers and their placements in the network by analysing average/worst case latencies. Lange et al. [8] addressed CPP in large scale and dynamic networks with the requirements of reducing the latencies (switch–controller and controller–controller), load balancing and fault tolerance. The authors proposed a Pareto Simulated Annealing (PSA) to identify the controller locations in the network.

Zhang et al. [9] addressed the CPP with the objective of achieving reliability, load balancing and minimizing the controller–switch latency. The experiment was conducted in MATLAB using the real-time topologies from the Internet topology zoo. They also presented the inference made regarding the impact of number of controllers in the network, network reliability, controller load factor and control path latency. Mouawad et al. [10] addressed the controller placement problem with the

aim to reduce communication latency, balanced load among controllers and to perform switch migration. The experiment was conducted on various topologies from the Internet topology zoo and analysed their proposal for various scenarios.

Liao et al. [11] addressed CPP with the objectives to reduce communication delay between the switch and the controller, address load balancing and analyse link failure cases. The proposed methodology, namely Density-based Switch Clustering Algorithm (DBSCA), uses density-based clustering algorithm to partition the network and to place controllers in the optimal locations. The load balancing algorithm was deployed as a SDN application in all the controllers placed in the network, and the switch migration was performed in the case of load imbalance condition.

Xiao et al. [12] addressed multicontroller placement for balancing the network load in Software-Defined Wide-Area Network (SDWAN). The authors deployed multiple controllers in the environment to balance load among controllers. To achieve balanced load, the authors proposed a Simulated Annealing Partition-based K-Means (SAPKM) in SDWAN environment. They defined two cost functions for load balancing indices and also considered the switch–controller communication latency, node degree of the switches, flow request rate on each controller to perform load balancing. The experiment was conducted on the real-time topologies taken from the Internet topology zoo, and the results were analysed.

2.2 Machine learning-based traffic classification in SDN

This section deals with the related works done in the traffic classification of SDN. The traffic classification methodology focussed more on finding anomaly traffic, providing QoS and classifying the applications. There are some works that focussed on SDNFV architecture to enhance the network performance.

In [13], the authors proposed a Decision Tree-based classification to classify the input traffic. The classified traffic was categorized into Elephant and Mice flows. A cost-sensitive approach was proposed to address the issue of traffic classification and prediction at low cost using decision trees. The proposed real-time elephant flow detection has two stages. In its first stage, to differentiate suspicious elephant flows from the mice flows, the authors analysed the packet measurement. In the secod stage, C4.5 decision tree is used to identify the distinguished packets are suspicious elephant flow or a normal elephant flow.

In [14], authors analysed both supervised and unsupervised machine learning model for classifying the application. The input data were collected from the OpenFlow statistics and concluded that the supervised machine learning algorithms performed better than the unsupervised ones. In [15], the supervised learning approach was proposed for data traffic classification in SDN networks. Naïve–Bayes, SVM and Nearest Centroid approaches were used for classification and their performance were analysed.

Indira et al. [16] analysed the network traffic using Deep Neural Network (DNN), a deep learning mechanism. The proposed mechanism classifies the incoming traffic with respect to low processing time and high accuracy. The authors have chosen ACL set (ACL_1K, ACL_5k, ACL_10k) to analyse the traffic. The authors

compared the proposed model with other similar models for performance metrics such as classification accuracy.

The controller performance was predicted by integrating AI and SDN [17]. The parameters taken for analysing the controller performance were round trip time, throughput and flow table content. A survey of machine learning techniques applied to SDN was briefly presented in [18]. This work also presented the future directions in which ML can be extended to improve SDN performance.

Hamdan et al. [19] proposed a flow aware elephant flow detection in SDN. The proposed methodology for classifying elephant flow used two classifiers, namely count-min technique and a Very Fast Decision Tree (VFDT) on switches and the controllers, respectively. Datasets used for traffic classification were MAWI, UNI1 and UNI2 for experimental purpose. The authors analysed the performance of the proposed using the performance metrics such as classification accuracy, precision, recall and F measure score.

2.3 SDNFV integrated architecture

In [20], the authors proposed the virtual platform for executing controller operations. Their proposed mechanism comprised two major components, namely Task Engine (TE) and Network Packet Scheduler (NPS), which were responsible for executing network functions. In [21], a dynamic network function on SDN and NFV environment in corresponding switches connected to controller. The proposed approach pre-deployed the appropriate function to be deployed in the switches. In [22], the authors proposed a OpenNetVM which discussed the benefits of integrating SDN and NFV which may deliver a smart control plane.

In [23], authors studied the machine learning algorithms such as C4.5, Bayesian networks, Decision Tree and Naïve Bayes to predict the network pattern. Their experimental studied depicted that the Decision Tree achieved highest accuracy in detecting the network pattern.

Ejaz et al. [24] proposed an integrated SDN and NFV environment to address load balancing in SDN. The authors converted the SDN controller as a virtual functionality (vSDN) and utilized it in the load imbalance scenario. The SDNFV orchestrator requests the SDN to create a virtual controller to balance the load in the network. The vSDN controller was created using the virtualization infrastructure manager (VIM) by creating a virtual machine to deploy vSDN. The authors used the Mininet emulator to conduct experiments.

Kwon et al. [25] proposed a methodology to address the traffic classification problem in SDN/NFV environment. The proposed mechanism proactively monitors, analyses the data automatically and predicts the network usage. The authors used supervised machine learning algorithms to analyse the network traffic. The Mininet emulator was used to create the SDN environment, and the traffic was collected using REST API from the SDN controllers. The supervised algorithms used were Random Forest (RF), Linear Discriminant Analysis (LDA) and Deep Neural Networks (DNN).

2.4 Summary of the literature

From the existing works, it is clear that the CPP has been addressed in various forms with respect to load balancing, reliable and multiple controller placements in the network. Most of the related works aim to reduce the communication latency between the switch and controller. Some works addressed load balancing among controllers. Few works addressed switch migration in the case of load imbalance in the network.

From the literature, it is clear that finding the number of controllers required with respect to traffic was not addressed. Though the initial placements of controller rely on the network topology, the identification of controller numbers and their optimal placements should be made according to the network traffic. The network traffic needs to be analysed for proper resource management. Few works in the literature tried to integrate SDN and NFV.

Very few works addressed the problem of load imbalance in the SDNFV environment. Though it balances the load, the placement of vSDN as NFV raises the problem of optimal placement of it. Therefore, optimal controller placements either in NFV or in the SDN control plane need to be addressed and also the proper utilization of it should be ensured. The application of ML in SDNFV environment was limited to network traffic classification. The network traffic-aware SDN controller placement in SDNFV environment was not explored much.

3 Problem statement

The separation of data and control plane from the traditional network devices introduces a problem in the placement of the software entities in the network. The SDN controller placed in the network has a significant effect on network performance. The placement of SDN controllers in their optimal locations of the network minimizes the flow installation time (FIT) of the packets which eventually reduces the network latency. The distance between the SDN controller and SDN switches decides the flow installation time. Therefore, the controllers should be placed in the optimal locations in the network. The placement of controllers highly depends on the network topology, but it becomes ineffective if that controller is not used frequently. Therefore, it becomes essential to consider the network traffic to ensure controller utilization and to study the controller's load.

The identification of controller number is variable that needs to be decided dynamically according to the network load. It cannot be a static one. The static number of controllers may not efficiently handle the network load, and it cannot be under-utilized or over-utilized. In order to identify the optimal number of controllers, the prediction of the network traffic becomes essential. Knowing the knowledge of future incoming network traffic may help in proper planning of the network. It may avoid unnecessary controller placements in the network. This emphasizes the optimal placements of controller in the network and the need for analysing the network traffic.

4 Proposed methodology

4.1 SDNFV architecture

The proposed architecture shown in Fig. 2 consists of SDN, NFV and an orchestrator to manage two network architectures. The proposed mechanism performs the optimal controller placements in the network by analysing and predicting the network traffic. This proposed mechanism also performs finding the required number of controllers to be placed in the network to manage the incoming traffic dynamically. It also performs network monitoring and performs network traffic classification.

NFV Infrastructure (NFVI) creates an environment to run all VNF deployed. It is responsible for holding the resources in the virtualized environment. The NFVI environment offers the computation components to perform processing, storage and provides link connections that connect to VNFs [24]. The NFV Management and Orchestration (NFV MANO) helps to establish connectivity between the SDN and NFV environments.

NFV MANO consists of three functional blocks, viz. NFV orchestrators which perform the integration of NFV with other environments, VNF managers to manage the VNF functionalities and virtualized infrastructure managers (VIM) which control and manages the NFV environment. The network functionalities are deployed in the Virtual Machine (VM) and are connected with the SDN environment via virtual links. Using NFV MANO, SDN and NFV are connected through virtual links and communicate with each other.

4.2 SDN controller placement

Given a network, $G = (V, E)$ represents an undirected graph where V stands for nodes and E stands for the links connecting the nodes. Let $V = \{V_1, V_2, V_3, \ldots, V_n\}$ where n stands for the number of switches in the network and C be the controller set $C = \{C_1, C_2, \ldots, C_k\}$, where k represents controller numbers in the network. The relation between the SDN switches (V) and the controller (C) can be defined as $C \subset V$ [26].

The initial controller selection process is done by using the initialization procedure of the K-Means++ algorithm. The initialization methodology initializes controller positions in the network and returns the number of controller placements. The numbers returned by the initialization method can be considered as the number of controllers required for the network. After initializing the controller numbers and their initial locations in the network, the K-Medoid algorithm tries to optimize the position by achieving the minimum SC communication delay. After executing the placement algorithm, the optimal controller locations are identified and placed in the network. The algorithm for controller placement methodology is defined in Algorithm 1.

Algorithm 1: Controller placement methodology

Input: G= (V, E), k, P (Placement set), $C = \{c_1, c_2, \ldots, c_k\}$

Output: Optimal Controller location (P)

// Initialize number of controllers and the initial positions in the network

1. Select the initial position c from C
2. for all switches i in V:

   $D(x) = \sum_{i=1}^{n} \|X_{c1} - X_{vi}\|$

3. Choose the next position of controller c_i, choosing $v \in V$ with the probability $\frac{D(x)^2}{\sum_{v \in V} D(v)^2}$
4. Repeat the steps until k initial controller positions are initialized
5. Return initialized 'k' controller numbers

// Identification of optimal controller placements in the network

6. for all initial controller position (j) in C_k controllers
7. for all SDN switches (i) in V
8. do
9. OPTIMAL:
10. if $V_i \notin C_j$
11. Swap V_i and C_j
12. $P_j = argmin \sum_{i=1}^{n} \sum_{j=1}^{k} \|X_{Vi} - C_j\|^2$
13. if P_j is minimal then
14. $P \leftarrow V_i$
15. i++
16. else
17. $P \leftarrow C_j$
18. goto OPTIMAL
19. j++
20. return P

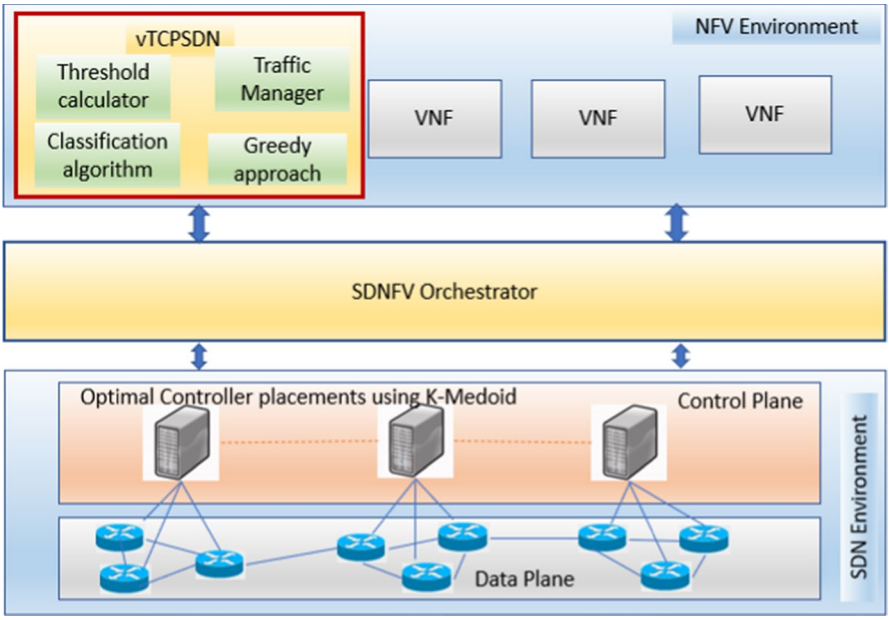


Fig. 2 Proposed architecture

4.3 Traffic analysis

The traffic analysis is performed by monitoring and analysing the network traffic periodically, and based on the analysis, the number of controllers required is identified dynamically and placed optimally in the network.

A LSTM-based framework is designed to analyse the network traffic. From the literature, it is evident that the LSTM delivers a promising solution to the complex problems in various scenarios and efficiently process a very large amount of data. As network generates voluminous data, LSTM would be suitable to transform the generated data into useful information. The main advantage of using LSTM is its memory features. The proposed methodology classifies the incoming traffic data into Low, Normal and High (LNH code). The following are the advantages of classifying the network traffic into LNH code.

1. First advantage is to identify the network traffic pattern.
2. Secondly, it helps to identify at what time, the controllers receive high traffic, low and normal traffic. It also helps to identify the idle time of the controllers.

3. Thirdly, based on the nature of traffic the number of controllers to be placed in the network can be identified. A network traffic pattern can be drawn based on this traffic classification. It also helps in deployment of appropriate number of controllers based on the incoming traffic when the network in process.
4. Finally, the proper utilization of controllers can be ensured and the controller's performance can be improved.

The proposed methodology is named as vTCP (vTraffic Classification and Prediction), and it performs traffic capturing, pre-processing of the incoming traffic and makes it suitable to the input to the LSTM, the traffic analysis and prediction of the network traffic. Figure 3 shows the work flow of the proposed model.

The dotted lines in the above figure show the functionality of the LSTM model, and it is deployed as a VNF and is accessed by all the controllers placed in the network.

4.4 Traffic generation and collection

Iperf, DITG and HTTP commands are used for network traffic generation. To prepare the dataset, the traffic manager collects the incoming traffic information using the Wireshark tool and also from the switches flow table.

4.5 Data pre-processing and feature extraction

In this step, the incoming traffic is pre-processed and make it suitable for the LSTM. The traffic manager collects the network traffic and performs data pre-processing. The dataset contains many fields which may not be suitable for performing the prediction of LNH code. The traffic manager starts the data cleaning process by removing the unwanted attributes.

The dataset preparation is an important step in any machine/deep learning models. In this experiment, the dataset was prepared by collecting flow statistics from the Wireshark tool and also from the switch flow table. The combined flow statistics (from Wireshark and switch flow table) contains many attributes which all may not necessary for this classification.

The initial dataset contains the attributes such as cookies, table (table number), idle_time, number of packets (n_packets), number of bytes (n_bytes), idle_timeout (flow expires after the specified seconds), idle_age, priority, protocol, vlan details, action commands from controller (to forward the packet/to

drop the packet), source and destination details, information about wildcard flow entry, duration (how many seconds the flow entry survives in the flow table) and time (at which the flow arrives). The data cleaning process is performed initially to remove unwanted attributes from the dataset. The dataset is created as follows.

The first field in the dataset is time, and the values are mapped to the corresponding time in order to understand how much traffic is been generated at that time. The N_Packets and N_Bytes are taken to identify how many packets and bytes are transferred. The dataset comprises various values, i.e. in different format which needs to be formatted or normalized to the range (0–1). The normalized values are given to the training process. The formula for normalizing the value is as follows:

$$z = \frac{(x - u)}{s}$$

where z is the normalized score, x is the value to be normalized, u is the mean and s is the standard deviation value.

To complete the dataset, the attributes controller and traffic are added. As the network is partitioned into domains, each domain is managed by a single controller and all the controllers have a unique port number through which their corresponding switches communicate. Using that port value, the values for controller attribute can be filled.

For an instance, suppose the switch port value is 6633, then the controller is named as A. Likewise, the controller values are filled and dataset is prepared. The attribute traffic value is calculated by using the attributes number of bytes and transferred. The final dataset which is to be sent to the LSTM contains the attributes such as Time, N_Packets, N_Bytes, Controller and Traffic (target attribute).

4.6 Learning phase of LSTM

The traffic manager sends the input dataset to the LSTM to learn and classify. The dataset is split into training and testing datasets. The 0.67 parts of the dataset is set for training the dataset, and the remaining 0.33 is set for testing. A new dataset was then given as the input to validate the performance of the LSTM. The model learns the condition for classifying the LNH code as in the algorithm given below.

Algorithm 2: vTCP mechanism

Traffic Classification and Prediction

Input: Traffic flow statistics

Output: Classification and Prediction of LNH code

// **Dataset preparation**

1. Traffic capturing using the Wireshark tool and by capturing of switch flow table.
2. Pre-process the input traffic by removing unwanted attributes and add the attributes controller and traffic to the dataset.
3. Fill the controller attributes value as stated in the data pre-processing step
4. Calculate the avg_mean value for the attributes N_Packets and N_Bytes and set up the threshold for classifying the LNH code.

// **Setting up the condition for classifying LNH code**

5. if ((N_Packets < avg_mean (N_Packets)) && (N_Bytes < avg_mean (N_Bytes)) then
6. assign: traffic ← Low
7. else if ((N_Packets == avg_mean (N_Packets)) && (N_Bytes == avg_mean (N_Bytes)) then
8. assign: traffic ← Normal
9. else
10. assign: traffic ← High

// **Classification**

11. Split the training and testing values as 0.67 and 0.33 respectively from the input dataset
12. Decide on the number of hidden layers and the hidden units of LSTM
13. Decide on the optimizer for each layer
14. LSTM learns to classify as follows.
    i. if the input value is lesser than the avg_mean, then it classifies the traffic data to Low.
    ii. if the input value is equal to the avg_mean, then it classifies as Normal.
    iii. if the input value is greater than the avg_mean, then it classifies as High.
15. The accuracy and the loss function are calculated for the predicted variables.
16. A new dataset is sent to the model to validate the performance and the performance metrics are studied.

The Recurrent Neural Network model with LSTM has emerged as the successful learning models for various applications involving time-series dataset. The key component of LSTM is its memory cells, and each cell consists of three gates namely: input, output and forget gates. The information flow is controlled by LSTM gates and prevents invalid operations on the LSTM layers [28] and thus makes it to store the data for the long term.

When the input dataset is given to the LSTM model, it performs the addition and regulation of the attributes. The input gate performs addition and regulation of input values. The forget gate improves the performance of the LSTM model by eliminating unnecessary flow information (low priority information) from the cell. The input to the forget gate is the value obtained from the previous hidden state (h_{t-1}) and the input of that particular time step (X_t). The input values are multiplied by the weight matrices, and a bias is added to it.

The Sigmoid function is applied to the result ranges from 0 to 1. If the resultant value is '0', that state has to be forgotten, and if it is '1', that state has to remember for further processing. The tanh function is applied in the output gate; a Sigmoid function is applied. The results from the Sigmoid are multiplied with the vector range of the previous step to draw the final output value. The procedure repeats until all the input values are processed. Using Eqs. (1–6), the LSTM gate values are calculated.

$$f_t = \sigma\left(W_f\left[h_{t-1}, X_t\right] + b_f\right) \tag{1}$$

$$i_t = \sigma\left(W_i\left[h_{t-1}, X_t\right] + b_i\right) \tag{2}$$

$$\widetilde{C}_t = \tanh\left(W_c\left[h_{t-1}, X_t\right] + b_c\right) \tag{3}$$

$$C_t = f_t * C_{t-1} + i_t * \widetilde{C}_t \tag{4}$$

$$o_t = \sigma\left(W_o\left[h_{t-1}, X_t\right] + b_o\right) \tag{5}$$

$$h_t = o_t * \tanh\left(c_t\right) \tag{6}$$

f_t is forget gate, i_t is input gate, o_t is output gate controls the output to the next hidden state, $\widetilde{C}_t$ and C_t are memory update functions, h_t refers to the output value, b_f, b_i and b_o are bias value.

4.7 Setting up of controller's threshold (C_T)

The traffic manager calculates the controller's threshold from the pre-processed dataset. The traffic manager calculates the controller threshold by analysing the FIT value. The FIT value is analysed for all PACKET_IN and PACKET_OUT messages. It identifies for which PACKET_IN message the FIT value starts increasing and it keep

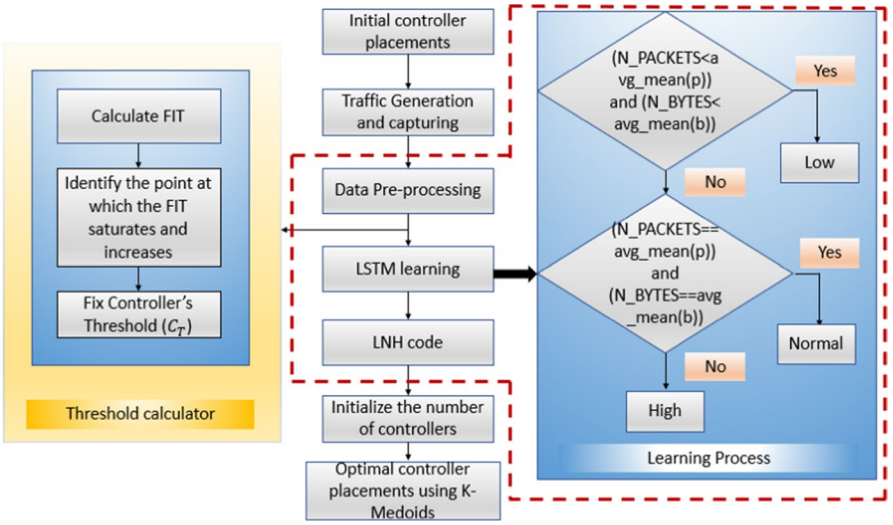


Fig. 3 Traffic analysis work flow

on increasing for the successive PACKET_IN messages. The FIT is the processing time of the controller upon receiving PACKET_IN message from the switch for computing the flow table value and send back resulting value as PACKET_OUT to the switch. The PACKET_IN message number in which FIT value increases is taken as the controller's threshold. As the network performance depends on the FIT latency, it is appropriate to consider FIT for setting up the controller's threshold (C_T).

4.8 Decision of number of controllers

Once the vTCP classifies the incoming traffic, the classified values are then sent to the traffic manager for further processing. The traffic manager performs the identification of number of controllers to be placed in the network by using Predicted Traffic (P_T) and the controller's threshold (C_T). Each and every controller to be placed in the network is considered as a bin. The controllers to be placed in the network are considered as a bin. The predicted traffic is stocked onto the controller till it reaches the controllers' threshold. Likewise, the predicted traffic is allotted to the controllers. The number of controllers (bins) filled is returned and is the considered as the number of controllers required to manage the network traffic. Algorithm 3 defines the methodology for calculating the number of controllers based on the predicted traffic. The calculated number of controllers is sent to the K-Medoid to place them optimally in the network.

Algorithm 3: Decision of number of controllers based on the predicted traffic

Input: Predicted Traffic (P_T), Controller Threshold (C_T)

Output: Number of controllers

```
Initialize the number of controllers (C_k) to 0.
while (P_T > 0)
 if (P_T > 0 && C_T < P_T )
 |  P_T = P_T − C_T
 |  C_k + +
 |  Continue;
 else
 |  C_k + +
 |  P_T = 0
Return C_k
```

5 Experimental setup and results

5.1 Experimental setup and performance analysis of vTCP SDN

This section describes the performance metrics taken for analysing the classifier and the obtained results. The vTCP SDN was made to run on the simulated cloud environment and it was configured in Linux by using NFV and Linux Containers. Traffic analysis algorithm is deployed in the container which acts as the cloud environment's VNF. It creates a virtual space where the proposed vTCP SDN methodology was deployed. The virtual link to communicate with the SDN environment was established.

The dataset contains all kinds of traffic (Low, Normal and High) and was taken from multiple runs. The total number of samples in the dataset is 135678 from which 0.67 (90904) is set for training and the remaining 0.33 (44773) is taken for testing the classifier. Once the dataset is ready for training the classifier, the required values of hyperparameters have to be set. The number of hidden layers and its hidden units are carefully chosen after multiple runs on the classifier.

The activation function used for each hidden layer is 'Relu', and for the final layer 'SoftMax' function was chosen. The Categorical Cross Entropy is taken for evaluating the loss function. The RMSprop optimizer was chosen as optimizer and the number of epochs is also set. The input traffic is classified into LNH code, and so the number of classes to be classified is 3.

5.2 Performance metrics to evaluate the vTCP SDN

The performance metrics taken here for evaluating the proposed vTCP SDN are precision, recall, F1 score, accuracy and categorical cross entropy.

The following Table 1 shows the formula for calculation of performance metrics.

The following Table 2 displays the metrics and the value obtained of vTCP SDN.

The following Figs. 4 and 5 depicts the performance of the proposed vTCP SDN in terms of accuracy and loss function.

The proposed methodology was compared with the multi-layer perceptron (MLP) with three layer (Input, hidden and output) and Auto-Regressive Integrate Moving Average (ARIMA) ($p=5$, $d=1$, $q=0$), and from the above figures, it is evident that the proposed methodology performs classification by observing the accuracy of 94.5%, whereas the MLP and ARIMA observe the accuracy 86.4% and 82.5%, respectively.

5.3 Experimental setup and performance metrics of controller placement

The SDN environment was created using the Mininet [29] emulator. The Mininet is the standard emulator for conducting SDN experiments. The topology for conducting the experiment was taken from the Internet topology zoo [30], a repository of real-time topologies. We considered each node in the topology as SDN switches, and the controllers can be deployed in any of the switch location. The SDN controllers communicate among themselves to maintain the global view of the network. The traffic was generated using Iperf, D-ITG commands and HTTP commands with randomly selected hosts. The traffic data were collected from the Wireshark tool and also from the switches flow table.

The placement algorithm was written in Python and was executed separately. After the optimal controller positions in the network were identified using the K-Medoid algorithm, those positions were marked as optimal controller locations in the Mininet topology and the switches were connected. Each switch in the network contained four hosts and the traffic was generated.

Figures 6, 7, 8 and 9 depicts the distribution of nodes and the position of controllers in the BTN, Forthnet, IRIS and LambdaNet, respectively. The x in the following figures represents the controller placements, and dots in the figures represent the nodes connected to the controllers.

The performance of the controllers is evaluated using FIT, Average Delay of the network and Throughput. The following Figs. 10, 11 and 12 depicts the FIT, average delay and throughput. The proposed methodology was compared with the K-Means++ algorithm [27], Pareto-Integrated Tabu Search (PITS) [26] and with the random placements (RP).

The proposed methodology for identification of optimal controller placements decreases the FIT by on an average of 32.2%, average delay by 28.3% and improves throughput by 22% shown in the following graphs.

The lowest FIT value was observed in the Sprint topology, and the highest value was observed in Bellsouth topology for the K-Medoid controller

placements. In Forthnet topology, same FIT value was observed for K-Medoid and K-Means++ controller placements. The values of FIT, average delay and throughput vary from topology to topology because of the topological structure and the distribution of nodes in each of the network topologies.

From the above graphs, it is evident that the proposed K-Medoid methodology for optimal controller placements outperforms the existing methodologies. The following Figs. 13, 14 and 15 depicts the performance of the controller placement with respect to FIT, average delay and throughput of the network before and after traffic prediction.

The controller placements were analysed before and after prediction of number of controllers using the K-Medoid analysed. The number of controllers was decided by using the greedy approach discussed in Sect. 4.5 used to identify the number of controllers needed to manage the incoming traffic. Once the number of controllers was decided, it was given to the proposed K-Medoid to identify the optimal locations in the network. Then, the network traffic was then generated and the results were analysed.

There is an average of 10–15% decrease in Flow Installation Time, which was observed after prediction of the traffic using the proposed K-Medoid method of controller placement. Likewise, 6–16% decrease in the average delay of the network was observed after prediction and 3–5% increase observed in the throughput of the network was observed using the proposed K-Medoid method of controller placement.

6 Conclusions and future enhancements

In SDN, SDN controller has the significant role in improving the network performance. As the SDN controllers perform all control activities, the placements of them affect the network performance. Therefore, the controllers must be placed in the optimal locations in the network. Though the controller placements depend on the network topology, it is quite essential to consider the network traffic to ensure proper utilization of controllers. A traffic-aware dynamic controller placement methodology was proposed to address CPP in the SDNFV environment. The proposed methodology identified optimal controller placements using K-Medoid algorithm and the performance of the placements were analysed through FIT, average delay and throughput. The proposed methodology was compared with the existing

Table 1 Performance metrics formula

No	Performance metrics	Formula
1	Precision	$\text{Precision} = \frac{T_p}{T_p+F_p}$
2	Recall	$\text{Recall} = \frac{T_p}{T_p+F_n}$
3	F1-score	$\text{F1} - \text{score} = 2 * \frac{(\text{precision} * \text{recall})}{(\text{precision}+\text{recall})}$
4	Categorical cross entropy	$\text{Loss} = \sum_{i=1}^{\text{outputrange}} y_i.\log\hat{y}$

where T_p—true positive rate, F_p—false positive rate, F_n—false negative rate

Table 2 Performance metrics of vTCP SDN

Traffic	Precision	Recall	F1-Score	Support
L	0.89	0.94	0.91	39,814
N	1.00	0.92	0.96	68,299
H	0.83	1	0.91	14,924
Accuracy	–	–	0.94	123,037
Macroaverage	0.91	0.95	0.93	123,037
Weighted average	0.94	0.95	0.94	123,037

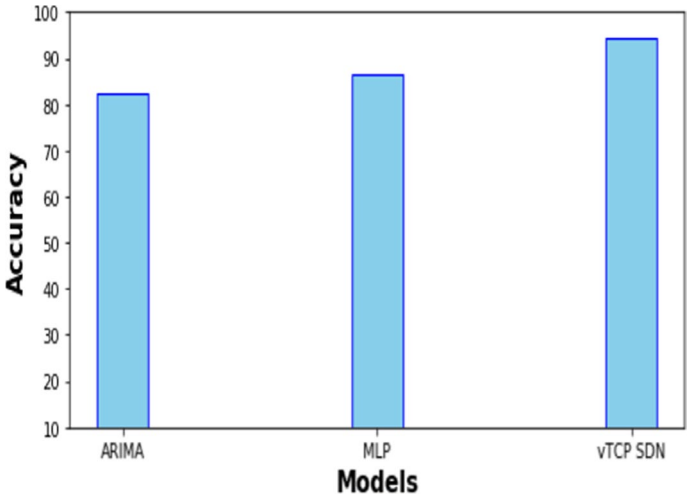


Fig. 4 Accuracy

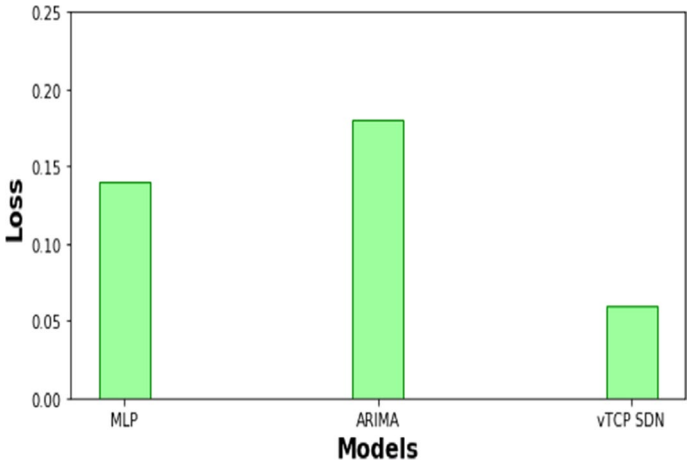


Fig. 5 Loss function

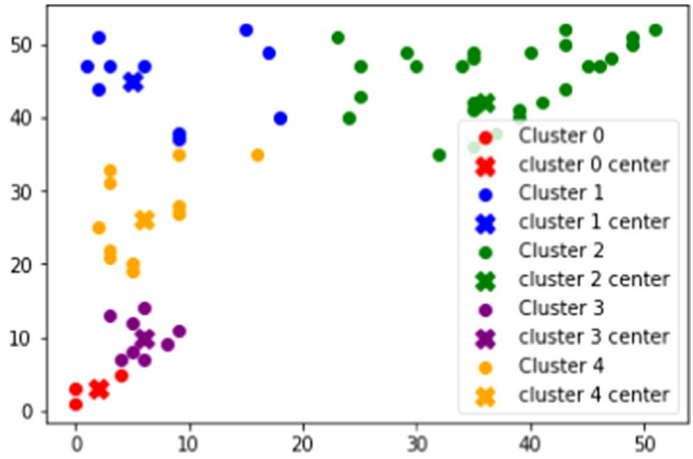


Fig. 6 Controller placement in BTN topology

methodologies, and from the observed results, it was evident that the proposed methodology successfully reduced the FIT, average delay and improves throughput significantly by 32.2%, 28.3% and 22%, respectively.

The vTCP SDN, network traffic analysis functionality was performed by using LSTM model, and the traffic analysis functionality was deployed as a VNF in the NFV environment using Linux containers. The performance of vTCP SDN was analysed using standard parameters such as precision, recall, F1 score, accuracy and categorical cross-entropy loss. The proposed methodology observed 94.5% of accuracy in the classification of LNH code. The experimental results validate that the

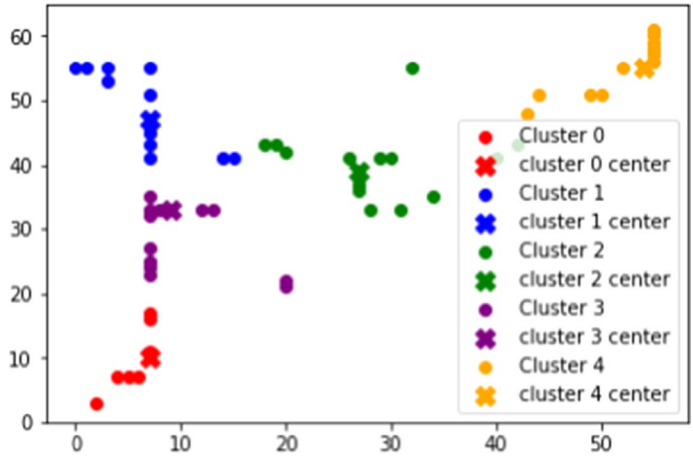


Fig. 7 Controller placement in Forthnet topology

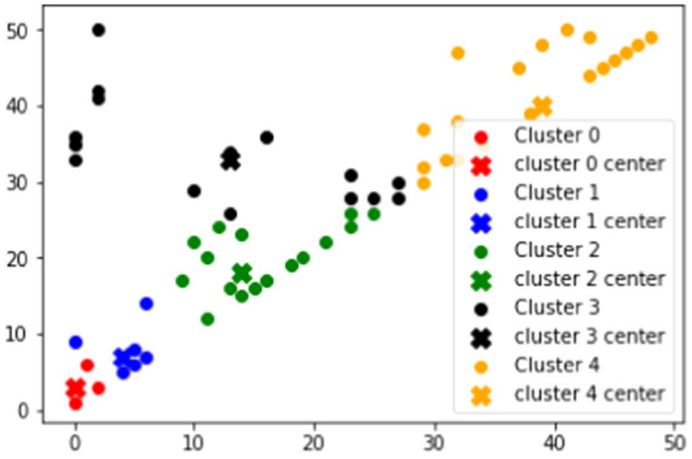


Fig. 8 Controller placement in IRIS topology

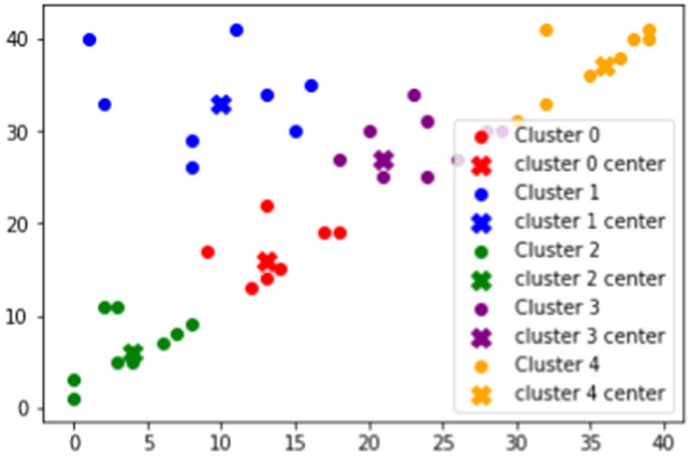


Fig. 9 Controller placement in LambdaNet topology

proposed methodology outperforms the existing methods and gives a new dimension to the CPP.

Though this work addressed a traffic-aware SDN controller placements in the network, it doesn't consider any of the failure cases. In future, traffic-aware reliable controller placement can be explored. The proposed vTCP SDN can be extended to identify network attack pattern. The proposed placement methodology for optimal controller placements can also be applied to the VNF placement in the NFV environment.

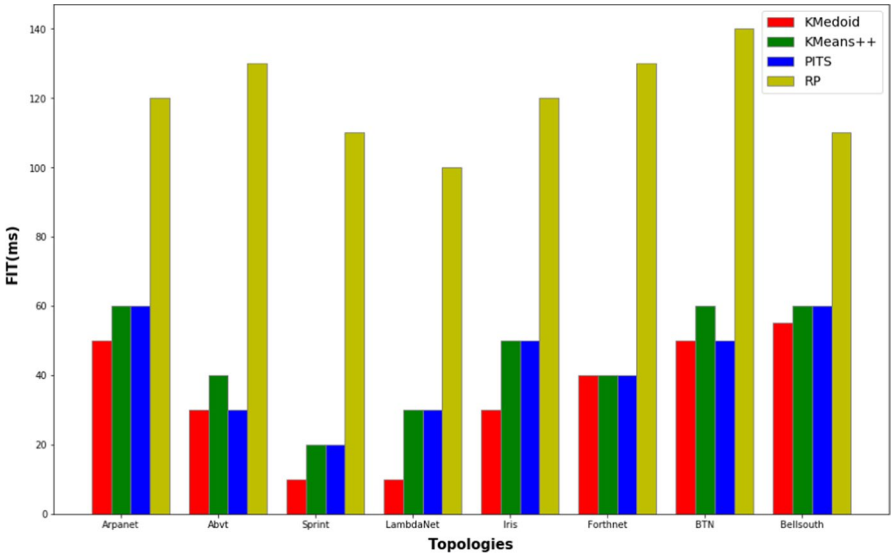


Fig. 10 FIT

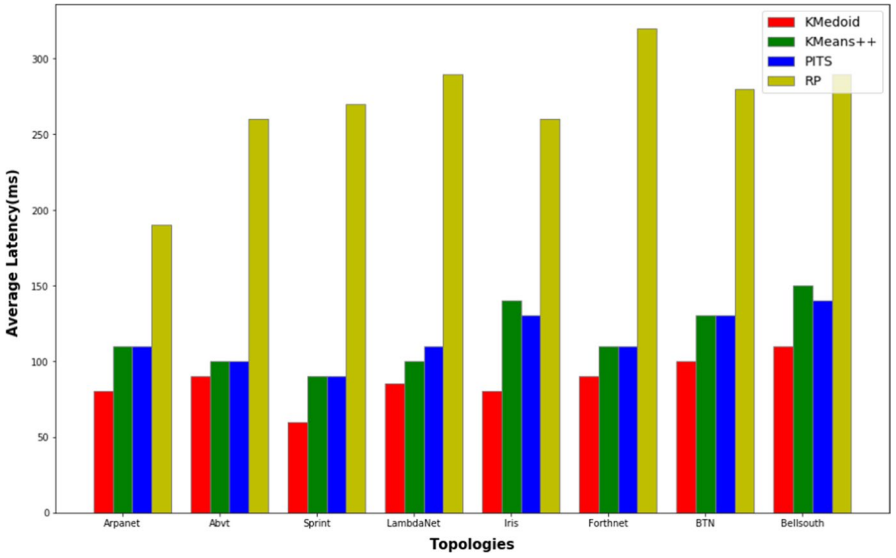


Fig. 11 Average latency

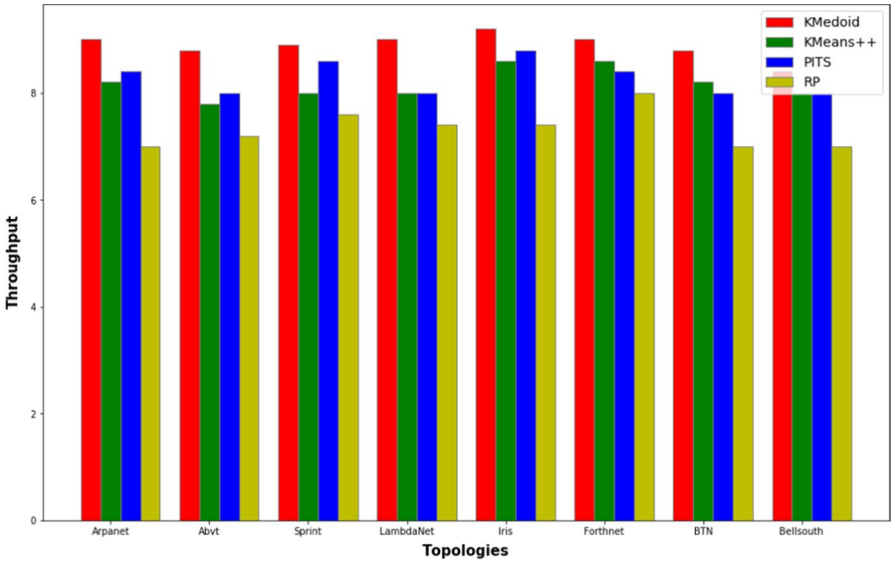


Fig. 12 Throughput

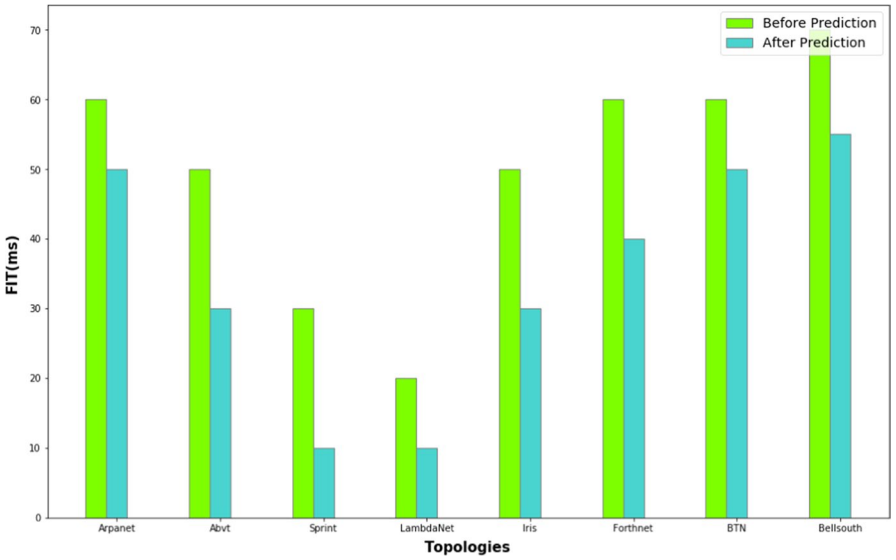


Fig. 13 FIT before and after prediction

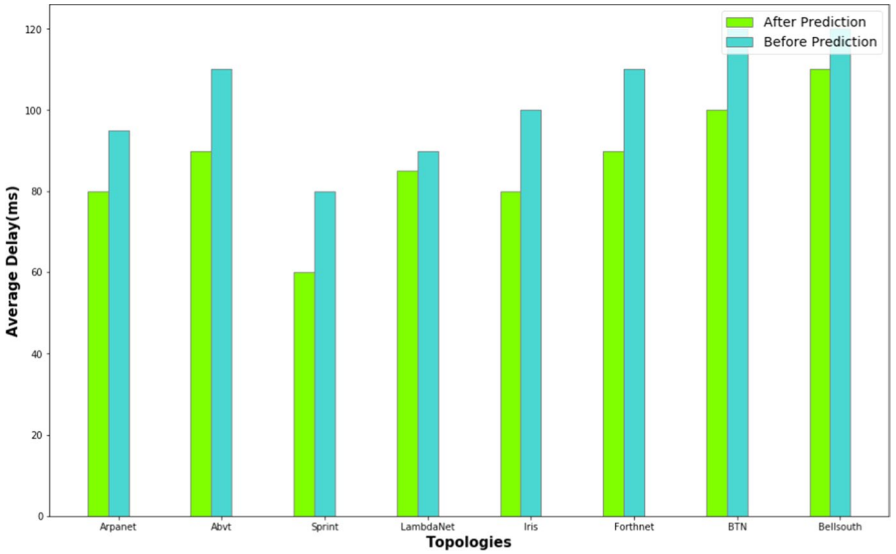


Fig. 14 Average latency before and after prediction

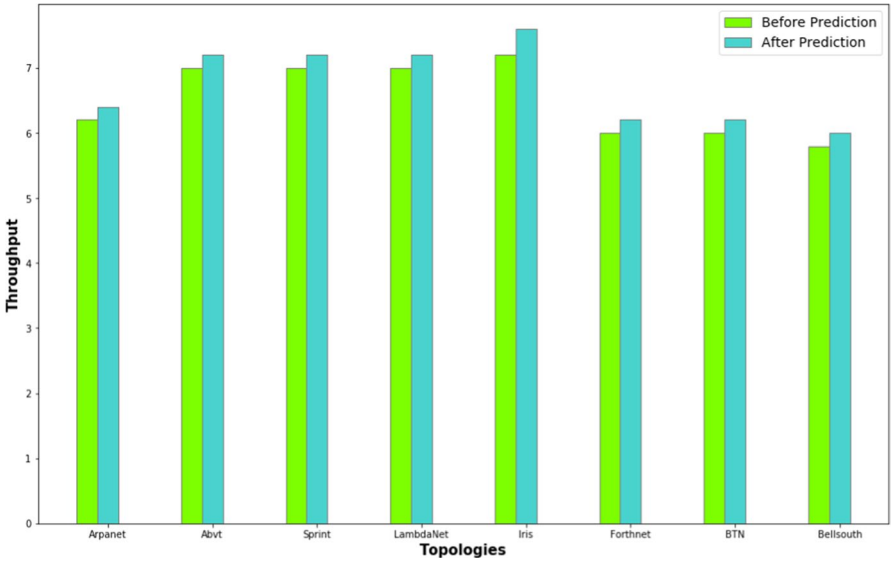


Fig. 15 Throughput before and after prediction

Data availability This manuscript contains no associated data.

Declarations

Conflict of interest There are no conflicts of interest to disclose.

References

1. Cisco U (2020) Cisco annual internet report (2018–2023) white paper. Cisco: San Jose, CA, USA, vol 10, no 1, pp 1–35
2. Farhady H, Lee H, Nakao A (2015) Software-defined networking: a survey. Comput Netw 81(4):79–95
3. Almadani B, Beg A, Mahmoud A (2021) Dsf: a distributed sdn control plane framework for the east/west interface. IEEE Access 9(1):26735–26754
4. Heller B, Sherwood R, McKeown N (2012) The controller placement problem. ACM SIGCOMM Comput Commun Rev 42(4):473–478
5. Kellerer W, Kalmbach P, Blenk A, Basta A, Reisslein M, Schmid S (2019) Adaptable and data-driven softwarized networks: review, opportunities and challenges. Proc IEEE 107(4):711–731
6. Xie J, Yu FR, Huang T, Xie R, Liu J, Wang C, Liu Y (2018) A survey of machine learning techniques applied to software defined networking (SDN): research issues and challenges. IEEE Commun Surv Tutor 21(1):393–430
7. Zhao Y, Li Ye, Zhang X, Geng G, Zhang W, Sun Y (2019) A survey of networking applications applying the software defined networking concept based on machine learning. IEEE Access 7(1):95385–95405
8. Lange S, Gebert S, Zinner T, Tran-Gia P, Hock D, Jarschel M, Hoffmann M (2015) Heuristic approaches to the controller placement problem in large scale SDN networks. IEEE Trans Netw Serv Manag 12(1):4–17
9. Zhang T, Giaccone P, Bianco A, De Domenico S (2017) The role of the inter-controller consensus in the placement of distributed SDN controllers. Comput Commun 113(1):1–13
10. Mouawad N, Naja R, Tohme S (2018) Optimal and dynamic SDN controller placement. In: 2018 International Conference on Computer and Applications (ICCA), vol 1, no 1, pp 1–9
11. Liao J, Sun H, Wang J, Qi Qi, Li K, Li T (2017) Density cluster-based approach for controller placement problem in large-scale software defined networkings. Comput Netw 112(1):24–35
12. Xiao P, Li Z-Y, Guo S, Qi H, Wen-yu Qu, Hai-sheng Yu (2016) A K self-adaptive SDN controller placement for wide area networks. Front Inf Technol Electron Eng 17(7):620–633
13. Xiao P, Qu W, Qi H, Xu Y, Li Z (2015) An efficient elephant flow detection with cost-sensitive in SDN. In: 2015 1st International Conference on Industrial Networks and Intelligent Systems (INISCom), vol 1, no 1, pp 24–28
14. Amaral P, Dinis J, Pinto P, Bernardo L, Tavares J, Mamede HS (2016) Machine learning in software defined networks: data collection and traffic classification. In: Proceedings of the IEEE ICNP'16, Singapore, vol 1, no 1, pp 1–5
15. Raikar MM, Meena SM, Mulla MM, Shetti NS, Karanandi M (2020) Data traffic classification in software defined networks (SDN) using supervised-learning. Proc Comput Sci 171(1):2750–2759
16. Indira B, Valarmathi K, Devaraj D (2019) An approach to enhance packet classification performance of software-defined network using deep learning. Soft Comput 23(18):8609–8619
17. Sabbeh A, Al-Dunainawi Y, Al-Raweshidy HS, Abbod MF (2016) Performance prediction of software defined network using an artificial neural network. In: 2016 SAI Computing Conference (SAI), vol 1, no 1, pp 80–84
18. Xie J, Yu FR, Huang T, Xie R, Liu J, Wang C, Liu Y (2018) A survey of machine learning techniques applied to software defined networking (SDN): research issues and challenges. IEEE Commun Surv Tutor 21(1):393–430
19. Hamdan M, Mohammed B, Humayun U, Abdelaziz A, Khan S, Ali MA, Imran M, Marsono MN (2020) Flow-aware elephant flow detection for software-defined networks. IEEE Access 8(1):72585–72597

20. Tripathy BK, Sahoo KS, Luhach AK, Jhanjhi NZ, Jena SK (2020) A virtual execution platform for OpenFlow controller using NFV. J King Saud Univers Comput Inf Sci 34(3):964–971
21. Bu C, Wang X, Huang M, Li K (2017) SDNFV-based dynamic network function deployment: model and mechanism. IEEE Commun Lett 22(1):93–96
22. Ramakrishnan KK (2016) Software-based networks: leveraging high-performance NFV platforms to meet future communication challenges. In: 2016 IEEE 36th International Conference on Distributed Computing Systems (ICDCS), vol 1, no 1, p 24
23. Nanda S, Zafari F, DeCusatis C, Wedaa E, Yang B (2016) Predicting network attack patterns in SDN using machine learning approach. In: Proceedings of the IEEE NFV-SDN'16, Palo Alto, CA, USA, vol 1, no 1, pp 167–172
24. Ejaz S, Iqbal Z, Shah PA, Bukhari BH, Ali A, Aadil F (2019) Traffic load balancing using software defined networking (SDN) controller as virtualized network function. IEEE Access 7(1):46646–46658
25. Kwon J, Jung D, Park H (2020) Traffic data classification using machine learning algorithms in SDN networks. In: 2020 International Conference on Information and Communication Technology Convergence (ICTC) vol 1, no 1, pp 1031–1033
26. Ramya G, Manoharan R (2021) Enhanced optimal placements of multi-controllers in SDN. J Ambient Intell Humaniz Comput 12(7):8187–8204
27. Ramya G, Manoharan R (2021) Prediction based dynamic controller placements in SDN. EAI Endors Trans Scalable Inf Syst 8(32):1–14
28. Wani A, Khaliq R (2021) SDN-based intrusion detection system for IoT using deep learning classifier (IDSIoT-SDL). CAAI Trans Intell Technol 6(3):281–290
29. Kaur K, Singh J, Ghumman NS (2014) Mininet as software defined networking testing platform. In: International Conference on Communication, Computing and Systems (ICCCS), vol 1, no 1, pp 139–42
30. Knight S, Nguyen HX, Falkner N (2011) Rhys Bowden and Matthew Roughan "The Internet Topology Zoo. IEEE J Sel Areas Commun 29(9):1765–1775

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Springer Nature or its licensor holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solely governed by the terms of such publishing agreement and applicable law.